

Academic year 2025-2026

**UE : IMA206 – Modèles génératifs, méthodes par
patches, photographie computationnelle**



Detection of synthetic images by spectral analysis

Mathilde **VIELLARD**

Adam **FAYRET-HOCHART**

Jules **DE**

ROLLAND-ALEXANDRE

Louis **BELLET**

Encadrant :

Quentin **BAMMEY**

Avril 2026

Table des matières

1	Use of Generative AI	2
2	Introduction	2
3	Theoretical part	3
3.1	Spectrum folding during resampling	3
3.2	Correlation between spectrum patches	3
3.3	A contrario detection and NFA	4
3.4	NFA computation	4
3.5	Ambiguity of the original size	5
3.6	Demo IPOL / Example of results and limits	5
4	Re-implementation	5
4.1	Pipeline	5
4.2	Preprocessing	6
4.3	2D - DFT	7
4.4	Interpolation filters	8
4.5	CNN to predict the interpolation filter	8
4.6	Implementation	9
4.7	NFA computation and Cross Validation	10
4.8	Difficulties encountered and limits	10
5	Integration and validation	11
5.1	Results and discussion	11
5.2	Estimation of the Resampling Factor	11
6	Extension : AI Detection	13
6.1	Towards other methods of detection	13
6.2	Conclusion on the other methods	13
7	Conclusion	14
A	Example of usage of AI	16

1 Use of Generative AI

Throughout this project we used generative AI tools in the following ways :

- **Intermediate tests** : The use of AI to design and generate test cases allowed us to validate intermediate results and check our progress. This helped us to iterate rapidly and identify potential issues early in the development process.
- **Plots** : AI helped us create visual representations of data, such as commented and precise graphs. This enabled us to present our results more effectively and explore different ways to illustrate our productions.
- **Debugging** : We also used AI to help us debug some python problems.
- **Translation** : We used AI to generate latex Algorithm out of our python implementation or to write efficiently tables.

Examples of those usage have been illustrated in Appendix ??.

2 Introduction

Images are often rescaled before uploaded to social networks or cropped for instance. Knowing whether an image has been resampled (and at which factor) is a key step in verifying its authenticity. For instance, apps and websites like Facebook, Instagram, Telegram, Twitter, and WhatsApp use JPEG-compressed images. The method suggested by the paper we studied proved to be more effective than other platforms across nearly all apps and websites. While other methods achieved about 1% precision at a 1% False-Positive Rate (FPR), the proposed method averages around 10% better. Only two websites resist this technique : Facebook and Twitter. Popescu achieved 8.6% accuracy in FPR, while the research paper reports 0.7% and 0.9%, respectively.

The original paper is *Image Resampling Detection via Spectral Correlation with False Alarm Control*. It proposes a method to detect images that have been resampled to then allow other analysis to detect if it has been AI-generated or not. Resampling is a common operation in image generation and forgery. But this method faces other problems, for instance we also know that compression such as JPEG can influence the spectrum by leaving some artifacts.

Objectives of this project

Starting from a theoretical analysis, we pursued two main objectives :

- Extend the implementation of the full detection pipeline from scratch in Python by calculating the resample factor.
- Explore methods to detect images generated by AI.

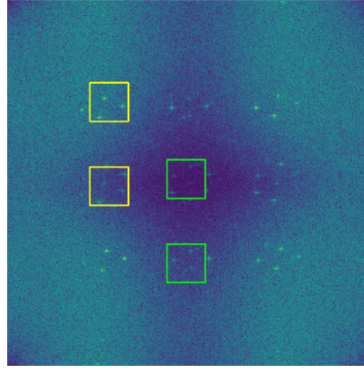


FIGURE 1 – Artifacts of resampled image

3 Theoretical part

3.1 Spectrum folding during resampling

Resampling goes through interpolation with filters such as bilinear, bicubic or Lanczos. The detection methods previously used were based on expectation-maximization, SVM, CNN or even LSTM. However they were not very effective or not very explainable so there was a need to do otherwise. The paper uses correlations in the Fourier domain and not the spatial one to identify the resizing.

From a set of samples $\{u_k\} \in \mathbb{R}^M$, interpolation is applied via a function φ to obtain a continuous signal f , which is then resampled to produce a new set $\{v_k\} \in \mathbb{R}^N$. The resampling formula is :

$$\tilde{v}_n = M \sum_{z \in \mathbb{Z}} c_{zN+n}(\varphi) \cdot \tilde{u}_{zN+n}$$

and shows how the Fourier coefficients of the interpolation filter modulate the original spectrum, resulting in a N-periodization of the initial M-periodic spectrum. This process is essential for adapting the resolution or sampling frequency of a signal while preserving its spectral characteristics.

The image 1 shows the spectrum of an image with some artifacts. That are highlighted in boxes. These spikes are what we are trying to analyze.

3.2 Correlation between spectrum patches

In practice, only one sample n is observed per frequency bin, making the estimation of $\text{Cov}[n, n+kM]$ unreliable. The paper instead measures the Pearson Correlation Coefficient (PCC) between small spectral patches $\tilde{w}_n$ and $\tilde{w}_{n+kM}$, each being the mean of over a small interval of length l centered at n (resp. $n+kM$). This averaging reduces estimation variance while preserving the correlation signal, since the covariance structure is approximately constant over a small frequency band.

3.3 A contrario detection and NFA

The a contrario method detects anomalies by calculating how many times an event would occur by chance. The hypothesis behind it is that the image is not resampled and thus, strong correlations are rare and the distributions are random. For each candidate distance d , it counts $k(d)$, the number of spectrum patch pairs whose correlation is a local maximum, and computes the Number of False Alarms which is the expected number of distances that would look this anomalous purely by chance. A detection is validated only when the probability of a peak is so unlikely under our hypothesis ...that it cannot be dismissed as a random fluctuation.

3.4 NFA computation

The a contrario detection relies on counting, for each candidate distance d , how many patch pairs exhibit a correlation peak at that distance. A correlation value $\rho(d)$ is defined as a *local maximum* if it is the maximum within a neighborhood of radius R :

$$\text{is_max}(d) = \mathbf{1} \left[\rho(d) = \max_{d' \in [d-R, d+R]} \rho(d') \right]$$

For each distance $d \in [d_{\min}, d_{\max}]$, we count $k(d)$, the number of patch pairs for which d is a local maximum of the correlation :

$$k(d) = \sum_{\text{patches}} \text{is_max}(d)$$

Under the null hypothesis $\mathcal{H}_0$ (no resampling), correlation peaks are uniformly distributed among the $2R + 1$ candidates in each neighborhood. The probability p that a given distance is a local maximum is thus $p_{\text{théo}} = \frac{1}{2R+1}$. In our implementation, we estimate p empirically from the data :

$$\hat{p} = \frac{1}{N_{\text{patches}} \cdot C} \sum_d k(d)$$

where $C = N_1 - 2R - 1$ is the number of tested distances. Under $\mathcal{H}_0$, $k(d)$ follows a binomial distribution $\mathcal{B}(N_{\text{patches}}, \hat{p})$. The Number of False Alarms is :

$$\text{NFA}(d) = C \cdot P(\mathcal{B}(N_{\text{patches}}, \hat{p}) \geq k(d))$$

A distance d is declared significant when $\text{NFA}(d) < \varepsilon$, with $\varepsilon = 10$ in our experiments.

Comparison with the reference implementation

Our implementation and the one from the repository differ in two key design choices, summarized in Table 1.

	Our implementation	Reference (git)
Definition of a peak	Topological event : local maximum in a neighborhood of radius R	Local maximum in a full correlation field, reconstructed from patch pairs
Probability p	Fixed : $\hat{p}$ estimated globally from all patches	Estimated per-distance from the data, adapted to the signal structure
Independence assumption	Patches treated as i.i.d. under $\mathcal{H}_0$	Dependence between patches accounted for via empirical estimation
Complexity	Fast, linear in the number of patches	Slower, depends on the image content and patch distribution

TABLE 1 – Comparison of NFA computation strategies.

Our approach follows a simplified i.i.d. model : it is computationally efficient and easy to interpret, but assumes that correlation peaks are statistically independent across patches. The reference implementation estimates probabilities from the actual patch correlations, making it more coherent with the real signal structure at the cost of higher computational complexity. In practice, both approaches detected the same resampling distances on our test images, validating our implementation despite its simplifying assumptions.

3.5 Ambiguity of the original size

...

3.6 Demo IPOL / Example of results and limits

The paper mentioned a website : <https://ipolcore.ipol.im>. This page was mostly used to run demonstrations on images. At first, we played with this tool in order to find the flaws. The algorithm behind the page managed to determine with unbelievable accuracy the nature of the image (resampled or original). However, we found some inputs that could put in a difficult position the demonstration. In fact, checkerboards or more generally images with high frequency variations would usually be detected as resampled than others less variations. Moreover, down sampled images were also likely to pass through the detection.

ANNEXE B ?

4 Re-implementation

4.1 Pipeline

Our Python re-implementation follows the pipeline of [2] :

1. Preprocessing (rank transform or TV denoiser).
2. 2-D DFT of the residual image.
3. Computation of patch-pair PCC scores for all distances d .
4. NFA computation and anomaly detection.
5. Bidirectional cross-validation assuming proportional resampling.

4.2 Preprocessing

1. Rank-Transformation (RK) : The rank-transform is a non-parametric and non-linear transform that replaces pixel values by their local ordinal rank, effectively homogenizing the spectrum and suppressing large-scale image structures. Our choice of implementation stands in [FIGURE]. We iterated over each pixel's neighborhood and counted how many pixels have an intensity value less than the central pixel, resulting in a rank-transformed image that highlights local intensity relationships. We chose to deal with the border problem with padding and to take a 7×7 neighborhood size which was a good compromise.

Algorithm 1 Rank Transform of an Image

```

1: Input : Image  $x$  (2D array of pixel intensities)
2: Output : Rank-transformed image  $rtImage$ 
3:  $image \leftarrow ConvertToFloat32(x)$ 
4:  $pad \leftarrow 3$  ▷ Half of the 7x7 neighborhood size
5:  $padded \leftarrow PadImage(image, pad, mode = 'reflect')$ 
6:  $H, W \leftarrow Shape(image)$ 
7:  $rtImage \leftarrow Zeros(H, W)$  ▷ Initialize output image
8: for  $x \leftarrow -pad$  to  $pad$  do
9:   for  $y \leftarrow -pad$  to  $pad$  do
10:    if  $y = 0$  and  $x = 0$  then ▷ Skip the central pixel
11:      continue
12:    end if
13:     $neighbor \leftarrow padded[pad + y : pad + y + H, pad + x : pad + x + W]$ 
14:     $rtImage \leftarrow rtImage + CastToFloat32(neighbor < image)$ 
15:  end for
16: end for
17: return  $rtImage$ 

```

We tested the implementation on both a table and images to take a look at the behavior of the function and the effect of it. We can see on [Figure] that the value returned by the algorithm is 1 for the minimum and 48 for the maximum because of the neighborhood size. This first test helped us better understand the results on images 2.

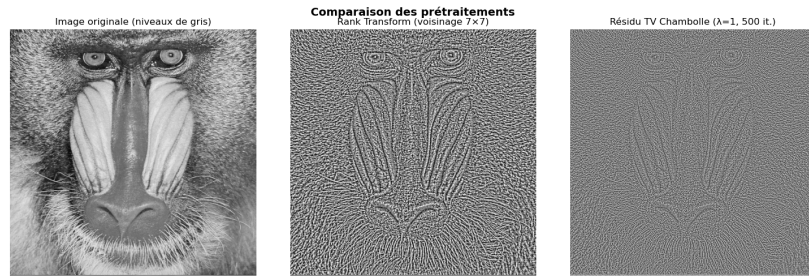


FIGURE 2 – Different pre processing

ALGO ET IMAGES DS UNE ANNEXE PRETRAITEMENT ?

2. Total-Variation (TV) : We also re implemented another pre-processing method which is very adapted for JPEG images. Total variation is a regularization that minimizes the sum of absolute gradients. It maintains sharp contrast and transitions. For our implementation, we applied TV with an iterative method and using the Chambolle algorithm.

Algorithm 2 Chambolle's projection algorithm for gray-scale TV-denoising

- 1: **Input** : A noisy image $f(i, j)$ as a $N \times N$ matrix, a trade-off parameter $\lambda > 0$, a time-step parameter $\delta_t > 0$ and an algorithm tolerance $tol > 0$.
 - 2: **Output** : The denoised image $u(i, j)$ as a $N \times N$ matrix.
 - 3: $p \leftarrow 0$
 - 4: **while** $\max_{1 \leq i, j \leq N} \{|p^{n+1}(i, j) - p^n(i, j)|\} > tol$ **do**
 - 5: **for all** pixel (i, j) in the image **do**
 - 6: $p(i, j) \leftarrow \frac{p(i, j) + \delta_t D(\text{div } p - \lambda f)(i, j)}{1 + \delta_t |D(\text{div } p - \lambda f)(i, j)|}$
 - 7: **end for**
 - 8: **end while**
 - 9: **return** $u = f - \frac{1}{\lambda} \text{div } p$
-

IMAGE ET ALGO DANS UNE ANNEXE PRETRAITEMENT ?

Comparison : For the project we continued with rank transform as preprocessing for PNG images because it is robust to noise and highlights structural details really well.

4.3 2D - DFT

To analyse the spectrum of image we use a 2-Dimensional Fast Fourier Transform (2D FTT) to analyse and get the Fourier Coefficient of our image. By simply plot the graph of the Fourier transform we can immediatly see artefact that we are going to analyse. For example in the baboon spectrum there are some constellations that are clearly identifiable. We are going to detect and analyze these bright points.

[IMAGE à INSERER : 4 images avec reconstruction]

4.4 Interpolation filters

Interpolating Filters are an important part in resampling as we often do not sample on points that already exist. We need to create the point that will be interpolated.

$$\{u_k\} \in \mathbb{R}^M \xrightarrow[\varphi]{\text{interpolation}} f \xrightarrow{\text{resampling}} \{v_k\} \in \mathbb{R}^N$$

The paper describe four different filters :

Interpolator	Expression of $\varphi(x)$	Fourier coefficients $c_n(\varphi)$
Nearest	$\begin{cases} 1, x \leq \frac{1}{2} \\ 0, \text{otherwise} \end{cases}$	$\frac{1}{M} \text{sinc}\left(\frac{n}{M}\right)$
Linear	$\begin{cases} 1 - x , x \leq 1 \\ 0, \text{otherwise} \end{cases}$	$\frac{1}{M} \text{sinc}^2\left(\frac{n}{M}\right)$
Cubic	$\begin{cases} \frac{3}{2} x ^3 - \frac{5}{2}x^2 + 1, x \leq 1 \\ -\frac{1}{2} x ^3 + \frac{5}{2}x^2 - 4 x + 2, 1 < x \leq 2 \\ 0, \text{otherwise} \end{cases}$	$\frac{1}{M} (3\text{sinc}^4\left(\frac{n}{M}\right) - 2\text{sinc}^2\left(\frac{n}{M}\right)\text{sinc}\left(\frac{2n}{M}\right))$
Lanczos	$\begin{cases} \text{sinc}(x)\text{sinc}\left(\frac{x}{3}\right), x \leq 3 \\ 0, \text{otherwise} \end{cases}$	$\frac{18}{M} (\text{Rect} * \text{Rect}(3\cdot) * \text{sinc}(6\cdot))\left(\frac{n}{M}\right),$ $\text{Rect}(\xi) := \begin{cases} 1, \xi \leq \frac{1}{2} \\ 0, \text{otherwise} \end{cases}$

We are analyzing some points on the spectrum that we found interesting (huge spike/bright points). Then we are analyzing different results for different filters, the filter that better match our observation on the spectrum is then the most likely used. We are implementing these filter eventhough they are not directly in the pipeline but only mention as a key factor in resampling. The implementation is a way to see their impact on the detection of resample images.

4.5 CNN to predict the interpolation filter

We built from scratch a convolutional neural network (CNN) which we trained to predict the resampling filter used in an image. This can be a powerful approach to address our question because resampling introduces different artifacts in images such as periodic patterns, aliasing, or unnatural correlations in pixel values. Those are often related to the filter or algorithm used. By training a CNN to classify the resampling filter, we can then use that information to go further in our investigation.

By combining filter prediction with other results we have, we can build a robust pipeline to :

1. Detects whether an image has been resampled.
2. Identify the filter or method used.
3. Compare and combine our different results to see if they are consistent with traditional resampling or not, suggesting AI generation.

4.6 Implementation

To train the CNN, we generated a synthetic dataset of resampled images. Each image was resampled using one of the following filters : "nearest", "bilinear", "bicubic", "lanczos", "nofilter". The resampling scale factors were randomly selected from a uniform distribution over the range [1.08, 1.90]. This range was chosen to simulate random resampling, we chose to exclude downsampling because the artefacts and effects are way harder to detect because the resampling causes less impact.

Pre-processing : Before feeding the images into the CNN, we applied a preprocessing pipeline to enhance the discriminative features related to resampling artifacts. The preprocessing steps included a rank transform and a feature extraction. We computed the normalized magnitude of the residual and cosine and sine components of the Discrete Fourier Transform (DFT) phase of the residual. These features were designed to capture both spatial and frequency-domain artifacts introduced by the resampling process.

CNN architecture : The CNN architecture was thought to progressively extract hierarchical features from the preprocessed images. The network consists of a series of 5 convolutional blocks, each followed by batch normalization (BN) and a ReLU activation function. Each ConvBlock consists of a convolutional layer, followed by batch normalization and a ReLU activation. This structure ensures that the network can capture both low-level and high-level features relevant to the resampling artifacts.

Results analysis : The performance of the CNN was evaluated using a confusion matrix and training/validation loss and accuracy curves. FIGURES The confusion matrix reveals that the CNN achieves high accuracy in classifying the resampling filters, with some confusion between similar filters. The loss and accuracy curves show that the model converges well, with training and validation metrics closely aligned, indicating minimal overfitting.

The proposed CNN, combined with the described pre-processing and dataset generation, effectively classifies resampling filters.

- **Dataset :** Resampled using "nearest", "bilinear", "bicubic", "lanczos", "nofilter".
- **Pre-processing :** Rank transform + normalized magnitude of residual + DFT phase components.
- **Architecture :** 5 convolutional blocks, batch normalization, ReLU activation.

4.7 NFA computation and Cross Validation

Cross Validation : Cross validation is a verification : when there is a real resampling, the factor q has to verify $N_1 = qM_1$ et $N_2 = qM_2$. When there is a false positive, the correlation only appears along one axis and we can eliminate it. Since there are calculus approximations we used the hypothesis $q_1/N_1 \approx q_2/N_2 \pm \beta$ (β is the relative tolerance). In order to do so we implemented it using a mask for each distance d_1 that identified the elements of NFA_cols which q is close to q_1 . We then took the minimal value found, which corresponds to the more conservative NFA to validate the significative. The function returns the validated NFA corrected with the number of tests in the margin.

In order to test the function we used a simulated scenario with two true motives (horizontal lines) and a false motive (noise). The Cross Validation function identified the true motives and the false ones.

PARAGRAPHE NFA

4.8 Difficulties encountered and limits

Modification Type	Test Status	Result (Distances d)	Technical Analysis & Interpretation
None (Original)	Success	No detection	Uniform and noisy spectrum. Expected behavior (no false positives).
Down-scaling ($\div 2$)	Failure (Mis-sed)	No detection	Image too small : few patches available (fragile statistics). The signal decreases but remains above the detection threshold.
Decimal Down-scaling ($\times 0.3$)	Failure	No detection	Search range becomes too narrow. The non-integer scaling factor spreads the correlation over multiple pixels.
Double Resampling	Failure	No detection	Successive transformations destroy periodic structures. The spectrum becomes uniform again.
Integer Up-scaling ($\times 4$)	Success	$d = [128, 256, 384]$	Excellent detection. Strong peaks corresponding to the fundamental frequency ($W/4$) and its harmonics ($\log_{10}(\text{NFA}) \approx -20$).
Integer Up-scaling ($\times 8$)	Partial Success	$d = [2, 6, 256]$	True detection at $d = 256$ (and one coherent harmonic), but two residual false positives remain ($d = 2, 6$).

TABLE 2 – Experimental results for different image scaling and resampling operations.

REDIGER PARTIE limited by nature for down-sampling Impacts from compression JPEG are similar to resampling effects

Difficulties : We encountered different difficulties during this phase of the project. From the concept described in the paper we had to make different choices for the implementation such as border problem and to understand implicit choices or concepts. Since we apply our codes to images we also had to master mathematical concepts in two dimensions and

to understand their limits and impact on the spectrum.

(In implementation of NFA ? ?)

5 Integration and validation

We then compared our own implementation with the one in GitHub linked to the paper. We had the same results for distance detection [Annexe A FAIRE POUR TOUTES LES PLOTS DE COMPARAISON]. We also tried the functions that they had coded in addition, as for instance different preprocessing. Those produced similar results which convinced us with our choice of rank transform.

We also test the optimization of the code using only parts of the image to do our calculation to see the impact. The results show that this technique allows a win of time without loss for the NFA.

However we could see that for instance they implemented the NFA in a quite different way. Our approach defines a pic as a topological event in a minima graph, using a fixed probability and assuming independence, which makes it fast but simplified. The GIT approach, however, defines a pic as a local maximum in a correlation field, estimates probabilities from the data, and aligns with the real signal structure, making it more accurate but slower and dependent on image patches. This new implementation is at the end more robust and coherent due to their hypothesis. [IMAGES en annexe]

5.1 Results and discussion

5.2 Estimation of the Resampling Factor

Principle

When an image of size L is resampled by a factor k , the resulting image has size $W = kL$. The a contrario detection identifies distances d at which abnormal correlations appear in the spectrum of the image of size W . Theoretically, these correlations occur at distances :

$$d_n = nL \bmod W, \quad n = 1, 2, 3, \dots$$

which correspond to successive foldings of the original spectrum of size L into the observed spectrum of size W .

For $n = 1$, we have $d_1 = L = W/k$. However, when $k < 2$, we have $d_1 = L > W/2$, so this peak is filtered out as it falls in the harmonic zone ($d > W/2$). What is observed in practice is its folded counterpart :

$$d^* = W - L = W \left(1 - \frac{1}{k}\right)$$

from which the resampling factor is directly recovered :

$$k = \frac{W}{W - d^*}$$

and the estimated original size is $\hat{L} = W - d^*$.

Domain of validity

This method is valid in the range $k \in]1, 2[$, for the following reasons.

Upper bound ($k \geq 2$). When $k \geq 2$, the fundamental distance $d^* = W(1 - 1/k) \geq W/2$ is filtered as a harmonic and can no longer be identified by this method.

Limitation : JPEG images

The method does not work reliably on JPEG-compressed images without additional precautions. JPEG compression introduces spectral artifacts at distances that are multiples of $W/16$, related to the 8×8 DCT block size. These spurious peaks can be confused with resampling peaks and either lead to false detections or mask the true d^* . The `is_jpeg` flag in the reference pipeline explicitly excludes these distances from the statistical test, but our factor estimator does not yet incorporate this filtering. INSERER IMAGE DE TEST SUR UN PANEL D'IMAGE

Experimental results

Image	detected d^*	estimated k	true k	error $ \Delta k $	
original (baboon.png)	—	—	1.000	—	
$\times 1.09$	48	1.094	1.090	0.004	
$\times 1.20$	102	1.199	1.200	0.001	
$\times 1.50$	256	1.500	1.500	0.000	
$\times 1.60$	307	1.600	1.600	0.000	
$\times 1.70$	358	1.699	1.700	0.001	
$\times 1.80$	410	1.801	1.800	0.001	
$\times 1.90$	461	1.900	1.900	0.000	
$\times 2.00$	—	—	2.000	—	
$\times 2.50$	—	—	2.500	—	

TABLE 3 – Resampling factor estimation results. Cases $k \geq 2$ are not detected as d^* falls in the filtered zone ($d > W/2$).

The results confirm the validity of the method across the full range $k \in]1, 2[$, with an error below 0.005 in all tested cases. For $k \geq 2$, no detection is produced, consistent with the theoretical analysis. Notably, the method remains accurate for $k \in]1.5, 2[$ despite the

harmonic ambiguity, because the fundamental peak d^* consistently produces a lower NFA than d_2 .

6 Extension : AI Detection

Continuum of the track : We wanted to push the project further and to think of ways to go from detecting whether an image has been resampled or not to, in the case of resampling, detecting whether it was generated by AI.

6.1 Towards other methods of detection

We have search for different methods to detect generation of images from AI. We have looked at SynthBuster and MaskSim. We have look to have the average spectrum of different models that are use for Image generation. As shown here [3](#).

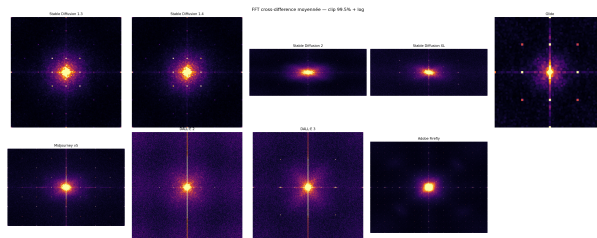


FIGURE 3 – Medium Spectrum of models

To get the average of these spectrum we implemented the Cross-Difference. That is a high-pass filter, to only detect the edge of the image. After that we compute the FFT of each image then we aggregate all the spectrum for each model. [4](#)

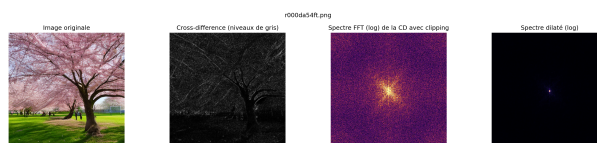


FIGURE 4 – Pipeline on one image

6.2 Conclusion on the other methods

In conclusion after being able to detect the resampling factor we went forward by trying to explore AI generated. To go even further we also looked at some results of the Masksim [\[1\]](#) and SynthBuster [\[3\]](#) paper. We manage to get an average spectrum for each model and to see the mask that was learn in MaskSim. That is our first step toward AI detection.

7 Conclusion

We have re-implemented the image resampling detection pipeline of [paper] from scratch gaining a thorough understanding of the spectrum folding phenomenon, the a contrario detection framework and the role of preprocessing in suppressing false positives. Despite some differences and lack of optimization, our implementation closely reproduced the results of the original code. Going further than the paper, we proposed and evaluated a method for estimating the resampling factor from the detected spectral peaks, and explored different based approach. These extensions remain challenging under strong JPEG compression or downsampling, which stays challenging fields for future work. We have also started to look into the generation of AI generated Images with the paper of SynthBuster and Masksim. As we believe such challenges will be important in the future.

Références

- [1] Y. Li, Q. Bammey, M. Gardella, T. Nikoukhah, J.-M. Morel, M. Colom, and R. Grompone von Gioi, “MaskSim : Detection of Synthetic Images by Masked Spectrum Similarity Analysis,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3855–3865, 2024.
- [2] Y. Li, Q. Bammey, M. Gardella, T. Nikoukhah, R. Grompone von Gioi, M. Colom, and J.-M. Morel, “Image Resampling Detection via Spectral Correlation with False Alarm Control,” *IEEE Transactions on Information Forensics and Security*, 2025.
- [3] Q. Bammey, “Synthbuster : Towards Detection of Diffusion Model Generated Images,” *IEEE Open Journal of Signal Processing*, vol. 5, pp. 1–9, 2024. 10.1109/OJSP.2023.3337714

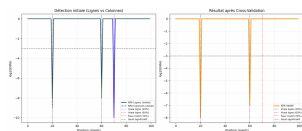


FIGURE 5 – Enter Caption

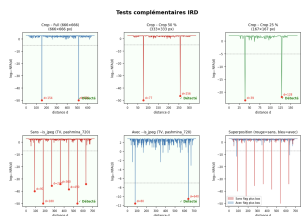


FIGURE 6 – Enter Caption

A Example of usage of AI

METTRE BIEN IMAGES + COMMENTAIRES SUR CE QUI A ETE GENERE ET LE CONTEXTE EN CAPTION

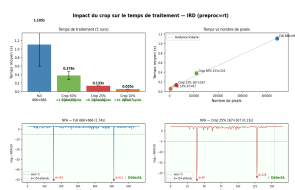


FIGURE 7 – Enter Caption

Algorithm 1 Rank Transform of an Image

```

1: Input: Image  $x$  (2D array of pixel intensities)
2: Output: Rank-transformed image  $rlImage$ 
3:  $image \leftarrow \text{ConvertToFloat32}(x)$ 
4:  $pad \leftarrow 3$  ▷ Half of the 7x7 neighborhood size
5:  $padded \leftarrow \text{PadImage}(image, pad, mode = 'reflect')$ 
6:  $H, W \leftarrow \text{Shape}(image)$ 
7:  $rlImage \leftarrow \text{Zeros}(H, W)$  ▷ Initialize output image
8: for  $x \leftarrow -pad$  to  $pad$  do
9:   for  $y \leftarrow -pad$  to  $pad$  do
10:    if  $y = 0$  and  $x = 0$  then ▷ Skip the central pixel
11:      continue
12:    end if
13:     $neighbor \leftarrow padded[pad + y : pad + y + H, pad + x : pad + x + W]$ 
14:     $rlImage \leftarrow \text{CastToFloat32}(neighbor < image)$ 
15:  end for
16: end for
17: return  $rlImage$ 
    
```

FIGURE 8 – Enter Caption